24je0609

Project_1

SHARED MEMORY

Implemented all the system calls of shared memory for IPC in xv6 OS.

```
shminit(void)
{
  initlock(&attachtable_lk, "attachtable");
  for (int i = 0; i < NSHM; i++) {
    initlock(&segtable[i].lk, "shmseg");
    segtable[i].used   = 0;
    segtable[i].refcnt = 0;
  }
  for (int p = 0; p < NPROC; p++)
    for (int s = 0; s < NSHM; s++)
      attachtable[p][s].shmid = -1;
}
```

```c
uint64
sys_shmget(void)
{
  int key, size;
  argint(0, &key);
  argint(1, &size);

  if (size <= 0)
    return -1;

  int npages = (size + PGSIZE - 1) / PGSIZE;
  if (npages > SHM_MAXPAGES)
    return -1;

  // If a segment with this key already exists, just return its id
  for (int i = 0; i < NSHM; i++) {
    acquire(&segtable[i].lk);
    if (segtable[i].used && segtable[i].key == key) {
      int id = i;
      release(&segtable[i].lk);
      return id;
    }
    release(&segtable[i].lk);
  }

  // Otherwise allocate a new slot and grab physical pages for it
  for (int i = 0; i < NSHM; i++) {
    acquire(&segtable[i].lk);
    if (!segtable[i].used) {
      for (int p = 0; p < npages; p++) {
        char *mem = kalloc();
        if (mem == 0) {
          for (int q = 0; q < p; q++)
            kfree((void *)segtable[i].pages[q]);
          release(&segtable[i].lk);
          return -1;
        }
        memset(mem, 0, PGSIZE);
        segtable[i].pages[p] = (uint64)mem;
      }
      segtable[i].used   = 1;
      segtable[i].key    = key;
      segtable[i].npages = npages;
      segtable[i].refcnt = 0;
      release(&segtable[i].lk);
      return i;
    }
    release(&segtable[i].lk);
  }

  return -1;
}
```

```c
uint64
sys_shmattach(void)
{
  int shmid;
  argint(0, &shmid);

  if (shmid < 0 || shmid >= NSHM)
    return (uint64)-1;

  struct proc *p = myproc();
  int pslot = pidslot(p->pid);
  if (pslot < 0)
    return (uint64)-1;

  acquire(&segtable[shmid].lk);
  if (!segtable[shmid].used) {
    release(&segtable[shmid].lk);
    return (uint64)-1;
  }
  int npages = segtable[shmid].npages;
  release(&segtable[shmid].lk);

  // Place shared region one guard page above the heap
  uint64 va = PGROUNDUP(p->sz) + PGSIZE;
  if (va + (uint64)npages * PGSIZE > TRAPFRAME)
    return (uint64)-1;

  acquire(&segtable[shmid].lk);
  for (int pg = 0; pg < npages; pg++) {
    uint64 pa = segtable[shmid].pages[pg];
    if (mappages(p->pagetable, va + (uint64)pg * PGSIZE,
                 PGSIZE, pa, PTE_R | PTE_W | PTE_U) != 0) {
      uvmunmap(p->pagetable, va, pg, 0);
      release(&segtable[shmid].lk);
      return (uint64)-1;
    }
  }
  segtable[shmid].refcnt++;
  release(&segtable[shmid].lk);

  // Remember this attachment so we can undo it in shmdetach
  acquire(&attachtable_lk);
  for (int s = 0; s < NSHM; s++) {
    if (attachtable[pslot][s].shmid == -1) {
      attachtable[pslot][s].shmid  = shmid;
      attachtable[pslot][s].va     = va;
      attachtable[pslot][s].npages = npages;
      break;
    }
  }
  release(&attachtable_lk);

  return va;
}
```

```c
uint64
sys_shmdetach(void)
{
  uint64 addr;
  argaddr(0, &addr);

  struct proc *p = myproc();
  int pslot = pidslot(p->pid);
  if (pslot < 0)
    return -1;

  // Find the record for this address
  acquire(&attachtable_lk);
  int found = -1;
  for (int s = 0; s < NSHM; s++) {
    if (attachtable[pslot][s].shmid != -1 &&
        attachtable[pslot][s].va == addr) {
      found = s;
      break;
    }
  }
  if (found < 0) {
    release(&attachtable_lk);
    return -1;
  }

  int shmid  = attachtable[pslot][found].shmid;
  int npages = attachtable[pslot][found].npages;

  // Clear our record first to prevent a double-detach
  attachtable[pslot][found].shmid = -1;
  attachtable[pslot][found].va    = 0;
  release(&attachtable_lk);

  // Remove the mapping from this process's page table (don't free the pages)
  uvmunmap(p->pagetable, addr, npages, 0);

  acquire(&segtable[shmid].lk);
  segtable[shmid].refcnt--;
  release(&segtable[shmid].lk);

  return 0;
}
```

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ shmtest
=== Shared Memory IPC Test ===
[parent] shmget: created segment with id=0 (key=42, size=4096)
[child]  attached segment at va=0x0000000000006000
[child]  wrote: "Hello from the child process!"
[child]  detached, exiting
[parent] child finished, now reading shared memory
[parent] attached segment at va=0x0000000000006000
[parent] read back: "Hello from the child process!"
[parent] detached successfully

=== Test PASSED: parent read the message written by child ===
$ 
```